Zephyr Project

IT-Schulungen.com
Technology & Transformation

Ankommen und Rückblick

Nebenläufigkeit im Zephyr-Kernel

2.01

Was wir in 15 Minuten klären

Gemeinsamer Ausgangspunkt für Threads, Scheduling und Kommunikation

01

Rückblick

Build, Run und Diagnose

02

main() als Thread

Der Haupt-Thread läuft bereits

03

Vier Ebenen

C-Code, Kconfig, Devicetree und CMake

04

Tagesbeispiel

Producer, Queue und Consumer

05

Startcheck

Build, Run und Projektstruktur

Am Ende ist die Ausgangsanwendung lauffähig und der Beobachtungsauftrag für Tag 2 klar.

Der Entwicklungszyklus

AUSGANGSPUNKT

Quellcode ändern und Erwartung formulieren

`Configure`	Kconfig und Devicetree auflösen
`Build`	Anwendung und Kernel erzeugen
`Run`	zephyr.exe als Linux-Prozess starten
`Observe`	Ausgabe, Timing und Zustände prüfen
`Diagnose`	Abweichung der passenden Ebene zuordnen

Vor jedem Lauf steht eine überprüfbare Erwartung.

Vier Ebenen, vier Aufgaben

LEITFRAGE

Welche Ebene erklärt das beobachtete Verhalten?

`C-Code`	definiert das Verhalten der Anwendung
`Kconfig`	aktiviert Kernel- und Subsystemfunktionen
`Devicetree`	beschreibt Geräte und Eigenschaften
`CMake`	ordnet Quellen und Buildstruktur zu

Zuerst die Ebene bestimmen, dann die passende Datei und das generierte Ergebnis prüfen.

Tagesbeispiel: Sensordaten-Pipeline

01	`Producer`	erzeugt reproduzierbare Messwerte
02	`Message Queue`	puffert Daten und macht Überlast sichtbar
03	`Consumer`	verarbeitet Werte und aktualisiert die Statistik

Kein Sensorboard nötig: native_sim macht Kernelverhalten reproduzierbar sichtbar.

Startcheck für Tag 2

BUILD

Anwendung bauen

Für native_sim bauen

Compilerfehler beheben

Kconfig-Warnungen prüfen

RUN

Anwendung starten

zephyr.exe ausführen

Erwartete Ausgabe erkennen

Prozess kontrolliert beenden

ORIENTIERUNG

Projektstruktur finden

main.c und prj.conf finden

app.overlay zuordnen

Build-Artefakte lokalisieren

Beobachtungsauftrag: Was ändert sich, sobald mehrere Threads gleichzeitig bereit sind?